Teacher:	Ray Joshua Balingkit	**Subject:**	Computer Science
Week:	5	**Year:**	10-11
Link:		**Cover:**	

LESSON OVERVIEW

What should students know/do by the end of this week's lessons?

Students will develop their understanding of computational thinking through decomposition and abstraction. They will learn how to break a complex problem into smaller, manageable sub-problems and how abstraction can be used to remove unnecessary detail when designing a solution. Students will also identify inputs, outputs, processing and initialisation requirements when analysing a problem and apply these techniques to a computer-based problem.

End-of-Week Assessment/Check: How will you know they got it?

Teacher questioning, class discussion, analysis of worked examples and the completed problem-solving activity. Students should be able to explain decomposition and abstraction in their own words, identify sub-problems within a larger problem and distinguish between input, output and processing requirements when analysing a problem.

LESSON SLOTS - MONDAY & FRIDAY

Lesson	Date	Learning Objective	The Lesson Flow	Differentiation
1	21 September 2026	By the end of this lesson, students will be able to explain computational thinking, decomposition and abstraction. They will be able to break a complex problem into smaller sub-problems and identify why decomposition makes a problem easier to solve.	**H:** Present a familiar complex problem, such as creating a computer version of a game. Ask: “If we had to program this entire game, could we solve everything at once?” Ask students to suggest smaller tasks that would need to be completed. **ID:** Introduce computational thinking and explain that computer scientists define and analyse problems before creating algorithms and programs. Introduce decomposition as breaking a problem into smaller, more manageable sub-problems. Explain that decomposition can make errors easier to identify and allows different parts of a problem to be worked on separately. **WD:** Examine the textbook's noughts and crosses example. Identify	**Support:** Provide a partially completed decomposition example and guide students through identifying the main problem and its smaller parts. Use the noughts and crosses example as a model and provide prompts such as “What does the user need to do?”, “What does the computer need to do?” and “How will the program know when it is finished?” **Extension:** Students decompose a more complex problem into a greater number of sub-problems and explain how different programmers or subprograms could work on the separate parts. Students who finish confidently can identify which sub-problems may need to interact with each other.

			the different sub-problems, including designing the 3×3 interface, tracking selected squares, deciding the computer's move and checking when the game is over. Discuss how solving these smaller problems contributes to solving the complete problem. **YD:** Students choose a familiar computer-based problem and decompose it into at least four smaller sub-problems. They should give a short description of what needs to be solved for each sub-problem. **C:** Ask selected students to share their decomposed problems. Exit check: students explain what decomposition means and give one benefit of using it when solving a programming problem.	
2	25 Septemb er 2026	By the end of this lesson, students will be able to explain abstraction and identify the essential details required to model a real-world problem. They will identify inputs, outputs, processing and initialisation requirements when designing a solution.	**H:** Show a real-world object or situation, such as a die or a journey. Ask: “If we wanted to create a computer model of this, would we need every detail of the real object?” Discuss which details are important and which could be ignored. **ID:** Introduce abstraction as removing or hiding unnecessary detail so that only the important points remain. Explain how programmers use abstraction when creating algorithms and models. Discuss the textbook's virtual die example and how only the essential behaviour of a real die needs to be represented. **WD:** Examine the noughts and crosses example and identify its inputs, outputs, processing and	**Support:** Provide a template divided into Problem → Sub-problems → Inputs → Outputs → Processing. Model one example before students begin. Provide vocabulary prompts for abstraction, input, output, processing, initialisation and subprogram. **Extension:** Students identify different levels of abstraction within the quiz game and explain which details can be hidden at a higher level. Students who finish confidently can write simple pseudocode describing how the wheel could select one of the four colours.

			initialisation requirements. Discuss how a programmer can work at a high level of abstraction before adding more detail. Introduce the idea of using separate subprograms for specific tasks. **YD:** Students analyse the textbook's Creating a Quiz Game activity. In groups or pairs, they list the requirements, decompose the problem into sub-problems and identify the inputs, outputs and processing requirements. Students should also describe how the spinning wheel could be modelled. **C:** Ask groups to present one part of their analysis. Exit check: students define abstraction in their own words and identify one input, one output and one processing requirement from the quiz game.	
3				Support: Extension:
4				Support: Extension:

MATERIALS NEEDED FOR THE WEEK

- Student Book / textbook pages 23–28
- Whiteboard / display screen
- Noughts and crosses example from the textbook
- Quiz Game activity from the textbook
- Problem decomposition worksheet / notebook
- Pseudocode template
- Computers, if students are completing the programming extension



END-OF-WEEK REFLECTION

- What worked well?

- What flopped or confused students?

- What needs to be re-taught or adjusted next week?